今月のフロントエンド

フロントエンド エンジニア集会 2026 年 10 月

"今月のフロントエンド" とは

今月あったフロントエンドのニュースを, 以下のフォーマットで紹介します.

- ニュースがあった技術について, その名前かキーワード
- その技術に関する解説 (3 行目安)
- 何があったかを解説

取り上げないもの

- 特定フレームワークに関するバージョンアップ (例外あり)
- AI 単品のニュース

VoidZero

2026/09/28



"VoidZero" とは

- Vue / Vite の作者 **Evan You** が 2024 年に設立した企業
- Vite / Vitest / Rolldown / Oxc など, フロントエンドのビルド基盤ツールを手がける
- 2026 年 6 月, チームごと **Cloudflare に合流** した

VoidZero のニュース

Vite+ 1.0 が正式リリースされた

- 9/28 公開. MIT ライセンスの OSS
- 依存している公開リポジトリは **2,600 以上**, 週 DL は **200 万目前**
- ただし本人たちも **「機能はまだ揃っていない」** と明言
  (リモートキャッシュや `vp release` などは今後の版で対応予定)

出典: Announcing Vite+ 1.0 (VoidZero)

"Vite+" とは

- VoidZero が開発する, フロントエンド開発ツールの **全部入りパッケージ**
- ビルド / テスト / コード検査 (Lint) / 整形を `vp` **という 1 つのコマンド** で扱う
- Node.js のバージョン管理やパッケージマネージャーの選択まで, `vp` が面倒を見る

Vinext

2026/09/28

"Vinext" とは

- Next.js (Vercel が開発する React のフレームワーク) のアプリを, **Vite の上で動かす** ための互換フレームワーク
- Cloudflare が開発. 既存の Next.js プロジェクトをそのまま動かせることを目指す
- 2026 年 2 月, Cloudflare のエンジニアが AI を使い **1 週間で作った実験** が出発点

Vinext のニュース

Vinext 1.0 がリリースされた

- 9/28 に公開
- Cloudflare は **「本番利用に耐える」** と位置づけた

出典: Next.js applications, powered by Vite: introducing Vinext 1.0 (Cloudflare Blog)

Next.js との互換

機能 (Next.js 16 系)	v1.0
App Router / Pages Router	○
Server Components / Server Actions	○
API ルート / ミドルウェア	○
Cache Components	△

- 主要機能のテストでの互換率は **99% 超**
  (Cache Components は集計の対象外)

出典: Next.js applications, powered by Vite: introducing Vinext 1.0 (Cloudflare Blog) / cloudflare/vinext

何が変わるのか

- Next.js は, 独自のビルドの仕組み (webpack / Turbopack) と一体で使うのが前提だった
- Vinext により, **Next.js のコードを Vite のエコシステムで動かす** 選択肢が本番レベルになった
- ただし全機能が揃ったわけではない
  → 移行は **自分のプロジェクトで動作確認してから**

IANA

2026/09/28

"IANA" とは

- インターネットの **番号と名前** を一元管理する機関 (ICANN の関連組織が運営)
- ドメインの最上位 (ルートゾーン), IP アドレスの割り当て, ポート番号などを管理
- 例示用に予約されたドメイン **example.com / example.net / example.org** を管理し,
  そのアクセス先のサイトも運営している

IANA のニュース

example.com のサイトを刷新 — 「テストや監視には使わないで」と明記

- 9/28, example.com などの予約ドメインのサイトを刷新した
- ページでは, **ドキュメントでの例示用** であり,
  テストや死活監視などの運用には使わない よう案内している

出典: IANA's email about why example.com changed (Oliver Dunk)

アクセスのほとんどは "自動アクセス"

- IANA によると, example.com へのアクセスの **大半は人ではなくプログラム**
- その出どころは, example.com を **「確実につながる宛先」** として扱うコード
  - テストコードの通信先
  - CI での疎通確認, 死活監視の宛先
  - サンプルをコピーしたまま残った設定
- **例示用の名前** が, 世界中で **インフラ代わりに使われてしまっている**
  → だからこそ「テストや監視には使わないで」と明記された

出典: IANA's email about why example.com changed (Oliver Dunk)

◯ フォームのプレースホルダー

メールアドレス




```
<input type="email" placeholder="you@example.com" />
```

- 入力例を見せる = **例示** そのもの. 本来の用途どおり
- 表示するだけで, どこにもアクセスしない

× テストコード

```
test("ユーザー一覧を取得できる", async () => {
  // NG: 実在する外部サイトにアクセスしてしまう
  const res = await fetch("https://example.com/api/users");
  expect(res.ok).toBe(true);
});
```

- 実害がなくても, テストが **外部へのアクセス** に依存してしまう
  → 相手の都合 (刷新, 停止, 応答の変化) で **テストが壊れる**
- そのアクセス自体が, IANA の負担になっている自動アクセスの一部になる

△ じゃあアクセスしなければいいのか？

```
test("ユーザー一覧を取得できる", async () => {
  // 通信はモックしているので，実際にはアクセスしない…はず
  vi.spyOn(globalThis, "fetch").mockResolvedValue(Response.json([]));
  const res = await fetch("https://example.com/api/users");
  expect(res.ok).toBe(true);
});
```

- 多用されると, どこかでミスを生む (特に AI が書き写す・書き換えるとき)
  → モックが外れた瞬間に, 実在するサイトへアクセスする
- アクセスの有無に限らず, **"例示" ではない用途は目的外**. 使うべきでない

まとめ — 例示なら OK, それ以外は NG

使い方	example.com
ドキュメント・サンプルコードの例示	◯
フォームのプレースホルダー	◯
テストコード (通信をモックしていても)	×
疎通確認・死活監視	× (言うまでもなく)

- テストで通信が必要なら, **モックするか, 自分で立てたローカルサーバー** に向ける
- 死活監視は, **自分で管理する宛先** に向ける

用途ごとの予約ドメイン (RFC 2606)

ドメイン	用途
`example.com` `example.net` `example.org`	ドキュメントでの例示
`.example`	ドキュメントでの例示
`.test`	テスト用
`.invalid`	**必ず無効** (失敗のテスト用)
`.localhost`	自分自身 (ループバック)

- テストでは **`.test`** / **`.invalid`** を使う

出典: RFC 2606: Reserved Top Level DNS Names (IETF)

JPEG XL

2026/10/06

"JPEG XL" とは

- JPEG の後継を狙って作られた, 国際標準 (ISO/IEC 18181) の画像フォーマット
- 同じ画質なら JPEG より **30〜50% 小さい**. 劣化のない圧縮や, 明暗差の大きい表示 (HDR) にも対応
- 既存の JPEG を **画質を落とさずに** JPEG XL へ変換し, 元に戻すこともできる

JPEG XL のニュース

Chrome と Firefox が, 標準で JPEG XL を表示できるようになった

- 10/6 公開の **Chrome 155** で, 実験フラグなしで表示できるようになった
- 10/13 公開の **Firefox 158** でも, 標準で有効化された
- これで **主要 3 エンジンすべて** が JPEG XL に対応した

出典: Shipping JPEG XL in Chrome (Chrome for Developers) / Firefox 158 release notes for developers

一度は消されていた

- Chrome は 91〜109 でフラグ付きの試験実装をしていたが,
  「エコシステムの関心が薄い」として Chrome 110 (2023 年) で削除 した
- その後, Safari 17 (2023 年) が先に対応
- 今回の Chrome は, **Rust で書き直したデコーダー (jxl-rs)** を採用
  → 画像の読み込み処理は攻撃の入り口になりやすく, 安全性を重視した

他ブラウザとの互換性

ブラウザ	JPEG XL の表示
Chrome	155〜
Safari	17〜
Firefox	158〜

- 非対応のブラウザでは **画像が表示されない** — 単に置き換えると壊れる
- `<picture>` で JPEG などの代替を併記すれば, **段階的に導入できる**

段階的に導入するには

対応ブラウザは JPEG XL を, 非対応ブラウザは JPEG を表示する

```
<picture>
  <source srcset="photo.jxl" type="image/jxl" />
  <img src="photo.jpg" alt="..." />
</picture>
```

Deno

2026/10/09

"Deno" とは

- Node.js の作者 **Ryan Dahl** が, 2018 年に発表した JavaScript の実行環境
- TypeScript をそのまま実行でき, ファイルやネットワークへのアクセスを **許可制** にできる
- ホスティングサービス **Deno Deploy** と, パッケージ置き場 **JSR** も運営している

Deno のニュース

Deno の開発チームが, まるごと Cloudflare に合流する

- **Deno 本体**: 今後 1 年間は, 月 1 回のバグ修正・セキュリティ更新を続ける
  → その後, **チームとしての開発は終了**. OSS として残し, 引き継ぎは歓迎するとした
- **Deno Deploy**: 6 か月間運用したのち **終了**. 有料顧客には移行支援
- **JSR**: 運営を続ける. 基盤は Cloudflare へ移す

出典: Deno is joining Cloudflare (Deno Blog)

Cloudflare で何をするのか

- Deno が開発していた, サーバー実行基盤 **celld** を Cloudflare Workers の OSS 版ランタイム **workerd** に統合する
- 狙いは, Cloudflare Workers と同じ仕組みを **自分のサーバーでも動かしやすくする** こと
- Deno 単体の実行環境とホスティングを続けるのではなく, **Cloudflare の基盤に力を集める** 判断

Cloudflare に集まる JavaScript の基盤

- **2026/06** — VoidZero (Vite / Vitest / Rolldown / Oxc) が合流
- **2026/09** — Vite+ 1.0 / Vinext 1.0 を公開
- **2026/10** — Deno が合流

いずれも OSS は維持するとしているが,
JavaScript の主要な基盤が **1 社に集まりつつある** ことは意識しておきたい

今月のインシデント

フロントエンド エンジニア集会 2026 年 10 月

"今月のインシデント" とは

今月あったフロントエンド周辺のインシデントを, 以下のフォーマットで紹介します.

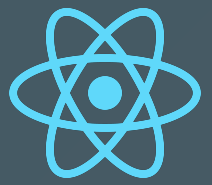
- インシデントの起きた技術について, その名前かキーワード
- その技術に関する解説 (3 行目安)
- 何が起きたかを解説

取り上げないもの

- フロントエンドと無関係なエコシステム単独の事案 (PyPI のみ, Go のみ など)
- 個別 CVE レベルの脆弱性の発見・修正
  (ただし, 規模が災害レベルなら例外的に取り上げる)
- SaaS / クラウド事業者単体への攻撃 (npm を経由しないもの)

React / Next.js

2026/10/14





"React" とは

- Meta (旧 Facebook) が開発した, UI を組み立てる JavaScript ライブラリ
- 画面を **コンポーネント** という部品に分けて組み立てる
- **React Server Components (RSC)** で, 一部の部品をサーバー側で動かせる

"Next.js" とは

- Vercel が開発する, React ベースのフルスタックフレームワーク
- サーバー側の処理まで含めて, React で Web アプリを作れる
- React Server Components を実用化した代表的な存在

2025 年 12 月に何が起きていたか

- React Server Components に, **認証不要で任意のコードを実行できる** 脆弱性が発覚
  (通称 **React2Shell**. 深刻度スコア CVSS で最高の 10.0)
- Next.js をはじめ, RSC を使うフレームワークが軒並み影響を受けた
- 公開直後から **実際の攻撃** が大規模に観測された

React / Next.js のインシデント

Next.js が臨時のセキュリティ更新 — 認証なしでコードを実行できる脆弱性が再び 2 件

- 最も深刻な Critical (CVSS 9.5) が 2 件, その次の High が 1 件
  - `use cache` 関数の処理で, **任意のコード実行** (RSC 側の問題)
  - 画像最適化の **SVG 処理** で, 任意のコード実行 (画像処理ライブラリの問題)
  - Server Actions で, サーバーを応答不能に追い込める (DoS 攻撃)
- 9 月の定例更新に入る予定が, **依存ライブラリ側の都合で延期** されていた

出典: Next.js Security Update for Upstream Vulnerabilities (Next.js Blog)

今すぐ確認すること

- Next.js は **16.4.1 / 15.5.28** 以上へ
- React の RSC 関連パッケージを直接使っているなら, **19.0.9 / 19.1.10 / 19.2.9 / 19.3.1** のいずれか以上へ
- `use cache` の問題には **回避策がない**. 使っていなくても更新が推奨されている
- 10/15 時点で悪用の報告はない — **攻撃が始まる前に** 塞いでおきたい

おわり